OpenClaw and Hermes Primer: A Comprehensive, Podman-First Guide

28 May 2026

Table of Contents

Long-form edition · 28 May 2026

---

1 Why This Primer Exists

When people first hear "OpenClaw," they can land on two completely different projects, and that ambiguity creates confusion before any technical work even starts. One historical usage points to an older game reimplementation, while the current, rapidly evolving project most practitioners mean is the OpenClaw AI assistant platform in the `openclaw/openclaw` repository. This document is explicitly about that modern assistant platform.

The second source of confusion is that the ecosystem around personal AI assistants has become noisy. Most guides either stay at marketing language or collapse into short install checklists that do not prepare you for real operation. In practice, the first five minutes are not the hard part. The hard part begins when you need to choose model routing rules, define tool execution boundaries, safely expose channels, manage persistent state, and recover quickly when something fails.

This primer is written to bridge that gap. It is intentionally long-form and operationally grounded.

1.1 What This Covers

By the end, you should have three things: a clear mental model of what OpenClaw is, practical patterns for how people actually use it, and a container-first setup path that keeps the host surface area as small and explicit as possible.

2 What OpenClaw Is Used For

At its core, OpenClaw is a personal assistant control plane that sits between users, channels, models, and tools. That sounds abstract until you map it to daily use: it is the system that decides how your assistant receives a message, which model should handle it, what tools are allowed to run, and where the final response should be delivered.

This distinction matters because many early "assistant" systems are really single-route pipelines: one UI talking to one model endpoint with minimal policy. That is often fine until you need multiple channels, fallback behavior, tool governance, or long-lived assistant state. OpenClaw is used specifically when those requirements become real.

Another practical reason people choose OpenClaw is that it supports a local-first posture without forcing a local-only posture. You can run local providers as your primary path for cost and privacy while keeping hosted providers configured as fallback for resilience. In practice, this balance is often more useful than ideological purity in either direction.

2.1 What jobs OpenClaw performs in practice

In real deployments, OpenClaw handles channel ingress and egress, session routing, model selection, provider failover behavior, tool-call policy, and operational checks. It also carries lifecycle responsibilities that are easy to underestimate: configuration validation, diagnostics, health status, and continuity across restarts.

That is why it is better viewed as an operations layer for assistants, not as a simple chat surface.

2.2 What OpenClaw is not

OpenClaw is not a model server, and it does not replace model-serving systems such as Ollama, LM Studio, or vLLM. It is also not merely a themed chat UI. Its value comes from orchestration and control, not from owning the underlying inference engine.

3 How People Actually Use OpenClaw

Successful OpenClaw usage tends to follow a few repeatable deployment patterns. The common trait across these patterns is disciplined boundaries: clear model policy, clear channel policy, and clear tool policy.

3.1 Pattern A: Single-user daily assistant

This is the best starting point for most users. One gateway, one main assistant identity, one or two channels, and simple model fallback rules. The benefit is not merely simplicity; it is diagnosability. When behavior goes wrong, you can identify cause quickly because there are fewer moving parts.

In this mode, OpenClaw usually acts as a practical command center for drafting, summarization, lightweight automation, and recurring workflows. Teams that skip this phase often end up debugging avoidable complexity later.

3.2 Pattern B: Multi-channel command center

Once the single-user baseline is stable, many users extend to multiple surfaces: Control UI, mobile nodes, and one or more chat channels. This is where OpenClaw's channel and session model becomes powerful. The same assistant can remain coherent across different delivery paths while preserving context and policy.

The security posture must evolve with this transition. Pairing rules, allowlists, and non-main sandboxing become core controls rather than optional hardening.

3.3 Pattern C: Local-first with hosted safety net

This pattern is increasingly common because it aligns cost, privacy, and reliability in a practical way. Local providers handle primary traffic. Hosted providers remain available as fallback when local services are unavailable, slow, or unsuitable for the request.

The result is a system that is more private than hosted-only, more resilient than local-only, and usually cheaper than always using cloud models.

3.4 Pattern D: Containerized operations

Operators who care about reproducibility and controlled blast radius often run OpenClaw in containers, keep state on explicit mounts, and use host-side CLI as the management plane. This is the posture emphasized throughout this guide because it matches a self-contained operational objective.

4 Ideas for How You Could Use OpenClaw

The most useful ideas are concrete enough that you can implement a first version in days, not months.

For documentation-heavy work, OpenClaw can become a documentation operations assistant that summarizes long markdown, drafts release narratives, and enforces style conventions. The practical payoff is reduced documentation drift and better continuity across fast-moving engineering work.

For engineering triage, OpenClaw can classify issues, suggest duplicates, and route work by subsystem while remaining constrained by explicit tool policy. Used carefully, it can reduce intake chaos without granting broad automation authority.

For personal research, OpenClaw can act as a persistent synthesis layer. The value is less about one perfect answer and more about retained context under storage boundaries you control.

For homelab operations, it can aggregate health checks and logs into digestible operational summaries. Even modest setups benefit when low-level telemetry becomes readable status rather than raw noise.

For role-separated workflows, OpenClaw can host multiple assistant identities with distinct workspaces and policy. That separation can drastically reduce accidental cross-context behavior.

5 OpenClaw Architecture in One Mental Model

A practical debugging model is to think in layers: gateway, agent, provider, execution, state. Most troubleshooting becomes easier when you identify the failing layer before changing configuration.

The gateway layer handles ingress, routing, APIs, and session plumbing. The agent layer carries prompt context, model selection logic, and tool-call behavior. The provider layer maps to model-serving endpoints and auth behavior. The execution layer is where tools run, either on host or sandbox. The state layer holds long-lived truth: config, auth profiles, session data, and workspace.

This layered view prevents category errors. A provider timeout is not a channel policy problem. A risky tool action is usually an execution-policy issue, not a model quality issue. A restart regression is often state drift, not immediate runtime logic.

5.1 State locations that matter

In container-first setups, state discipline is non-negotiable. Configuration, auth profile material, workspace data, and session artifacts should all persist outside ephemeral container layers. If this boundary is unclear, upgrades and restores become fragile.

6 Comprehensive Local Setup (Podman-First, Self-Contained)

This section is intentionally operational and assumes your goal is repeatable operation, not one-time demonstration.

6.1 Deployment goals

A strong target posture is rootless Podman runtime, explicit state persistence mounts, minimal host dependencies, and optional user-level service management for restart behavior. This keeps host contracts narrow while preserving operational control.

6.2 Prerequisites

You need Linux, rootless Podman, OpenClaw CLI on host, and optionally `systemd --user` for service management. On headless systems, lingering can be used for boot-time continuity.

6.3 Bootstrapping flow

Use source checkout to align with official helper scripts.

```
git clone https://github.com/openclaw/openclaw.git
cd openclaw
```

Initialize Podman path:

```
./scripts/podman/setup.sh
```

Launch runtime:

```
./scripts/run-openclaw-podman.sh launch
```

Run onboarding in container context:

```
./scripts/run-openclaw-podman.sh launch setup
```

Access dashboard:

- `http://127.0.0.1:18789/`

Operate via host CLI targeting the container:

```
export OPENCLAW_CONTAINER=openclaw
openclaw gateway status --deep
openclaw dashboard --no-open
```

6.4 Persistence model

Treat persistence as architecture, not convenience. Config, workspace, auth, and session artifacts should all map to known durable paths. Avoid anonymous state where possible.

6.5 Optional Quadlet mode

If you need service semantics and restart behavior, user-level Quadlet can provide cleaner day-2 operations than manual relaunch loops.

6.6 Day-2 operations

```
podman logs -f openclaw
podman stop openclaw
./scripts/run-openclaw-podman.sh launch
openclaw gateway status --deep
openclaw doctor
```

6.7 Ollama-native quick setup

The Podman bootstrapping flow above is the self-contained posture this guide emphasizes, but there is a faster path for users who already run Ollama and want a single-command launch. Ollama can drive OpenClaw directly, handling installation, model selection, and daemon startup in one step.

```
ollama launch openclaw
```

On first run this walks you through the full interactive setup:

1. Installing OpenClaw via npm if it is not already present.
2. A security notice explaining the tool-level access the agent will be granted.
3. Model selection — local or cloud.
4. Configuring your messaging provider(s) and starting the gateway daemon.

For unattended starts — boot-time services or container launch — use the headless variant. `--yes` skips the interactive prompts and `--model` is required:

```
ollama launch openclaw --model qwen3.5 --yes
```

To stop the gateway:

```
openclaw gateway stop
```

This path trades some of the explicit container boundaries described above for convenience. It is a good fit for single-user local setups where you control the host directly.

6.8 Recommended Adoption Sequence

1. Bring up Podman runtime and verify health.
2. Configure one local provider first.
3. Add one hosted fallback.
4. Enable non-main sandboxing before opening external channels.
5. Containerize model services for stronger containment if needed.
6. Establish backup cadence.

7 Running OpenClaw with Local LLMs

OpenClaw integrates with both native local providers and OpenAI-compatible proxy-style providers. Choosing between them is primarily about behavior guarantees and operational preference.

7.1 Model selection and fallback semantics

OpenClaw distinguishes configured defaults, auto-selected fallback state, and explicit user overrides. This is operationally important. Configured defaults can walk fallback chains.

Explicit user selections are strict by design and fail visibly when unavailable.

Model choice is not just a quality decision; it is a context-budget decision. OpenClaw is an agentic assistant that does multi-turn reasoning, calls tools, and processes long context, so the local model you pick needs room to work. Plan on at least a 64K token context window for local models running agentic workloads. Agent loops accumulate tool call results, conversation history, and web search output into context very quickly, and a model that cannot hold that working set will start truncating or failing mid-task.

The following models are practical defaults for local-first operation, with two cloud entries kept as fallback:

Model	VRAM needed	Notes
`qwen3.5` (local)	~11 GB	Reasoning, coding, vision — the local sweet spot
`gemma4` (local)	~16 GB	Strong reasoning and code
`qwen3.5:cloud`	None local	Falls back to Ollama cloud; good for testing
`kimi-k2.5:cloud`	None local	Multimodal reasoning with sub-agents

7.2 Ollama

Ollama is a strong local-first path, but the key setup detail is API mode. For OpenClaw's Ollama provider, native API endpoint behavior is preferred over `/v1` compatibility mode when reliable tool behavior matters.

```
ollama pull gemma4
export OLLAMA_API_KEY="ollama-local"
openclaw onboard
openclaw models list --provider ollama
openclaw models set ollama/gemma4
```

7.3 LM Studio, vLLM, and LiteLLM

These three are all reached through OpenClaw's generic OpenAI-compatible provider path — the "Generic OpenAI-compatible local provider" config later in this primer is the concrete stanza to adapt for any of them, pointed at each backend's own base URL and API key.

LM Studio is useful when you want local model serving with easier lifecycle controls than raw llama.cpp — its GUI handles model download and switching, and it exposes an OpenAI-compatible endpoint OpenClaw can target directly.

vLLM is commonly used for higher-throughput serving scenarios, particularly when serving one model to multiple concurrent agents or users. In OpenClaw, it is treated as an OpenAI-compatible provider and should be configured with explicit timeout and model metadata assumptions, since vLLM's own defaults are tuned for throughput rather than agentic tool-calling latency.

LiteLLM is valuable as an abstraction and routing layer over multiple model backends — it fronts several providers (local and hosted) behind one OpenAI-compatible endpoint, so it's often used where centralized policy and provider switching are required rather than talking to a single backend directly.

7.4 On-demand local services

OpenClaw can also manage provider-local service startup via `localService` config, allowing heavyweight model services to spin up on demand instead of running continuously.

7.5 Constrained hardware: partial GPU offloading

The reason to run this on constrained hardware at all is not raw speed — you will not beat a hosted model on tokens per second. The benefit is privacy and persistence: local files, local databases, and MCP-connected tools, all under boundaries you own. On a 64 GB RAM / 6 GB VRAM laptop this is enough to run a practical roaming assistant with large retained context, which is often more valuable day-to-day than a faster model that forgets everything between sessions.

That class of laptop cannot fully host the recommended OpenClaw models in 6 GB of VRAM, but 64 GB of RAM is plenty for a strong partial-offload setup: put as many layers as possible on the GPU and keep the rest on CPU/RAM.

With llama.cpp, the `--n-gpu-layers` flag controls how many transformer layers go to the GPU. Layer counts vary by model family: Qwen2.5-7B has 28 transformer layers, while Llama-family 7B models have 32. Loading 22 of Qwen2.5-7B's 28 layers onto the GPU typically uses ~3.5–4 GB of VRAM for weights, with the remaining 6 layers running on CPU/RAM — a genuine partial offload, not a full one.

KV cache doesn't follow the weights split the way you might expect. In llama.cpp and Ollama, the KV cache for GPU-offloaded layers lives in VRAM by default, not RAM — only the CPU-resident layers' KV cache lives in system RAM. So your 64 GB of RAM absorbs KV-cache growth for the 6 CPU-resident layers, but VRAM still has to hold the growing KV cache for the 22 GPU-resident layers as context fills. At a 64K context window, that VRAM-side KV

cache can add several GB on top of the ~3.5–4 GB of weights, which will not fit comfortably in 6 GB. Squeeze it down with KV cache quantization (`OLLAMA_KV_CACHE_TYPE=q8_0`, or `q4_0` if you need more headroom) rather than assuming the full fp16 KV cache is free.

With Ollama, set the equivalent through a Modelfile:

```
export OLLAMA_KV_CACHE_TYPE=q8_0

cat > ~/qwen-laptop.Modelfile << 'EOF'
FROM qwen2.5:7b-instruct-q4_K_M
PARAMETER num_gpu 22
PARAMETER num_ctx 65536
PARAMETER num_thread 8
EOF

ollama create qwen-laptop -f ~/qwen-laptop.Modelfile
ollama launch openclaw --model qwen-laptop
```

Expect roughly 8–15 tok/s for 7B Q4 with partial offload on a modern Intel/AMD laptop. Long-context prefill is slower, but interactive chat stays usable. Treat `num_thread 8` as a starting point and tune toward your physical core count — too many threads adds overhead rather than throughput.

8 OpenClaw vs Hermes: What Is Similar and What Is Different

OpenClaw and Hermes are often mentioned in the same conversations because they can be used together in the same local-first stack. The overlap is real, but they sit at different layers and solve different problems.

At a high level, OpenClaw is an assistant orchestration system. Hermes is a model family. If you treat those as interchangeable, architecture decisions become blurry very quickly.

8.1 Core distinction: system layer vs model layer

OpenClaw is the control plane: it routes channels, applies policy, manages tool permissions, handles provider fallbacks, and coordinates state over long-running sessions.

Hermes is the inference engine option inside that control plane: it turns prompts into completions and, depending on model generation and prompt format, can produce structured tool-call outputs.

In short, OpenClaw decides how work is run. Hermes helps perform one reasoning step at a time.

8.2 Similarities that make people compare them

The comparison is not random. In practical local deployments, both are used in support of the same goals:

1. Local-first control over data and model execution.
2. Reduced dependence on hosted APIs for daily assistant tasks.
3. Better auditability than opaque managed workflows.
4. Strong fit for coding, operations, and long-form technical workflows.

That shared use case is why people often say "OpenClaw vs Hermes" even though the more precise framing is "OpenClaw with Hermes".

8.3 Biggest differences in practice

Dimension	OpenClaw	Hermes
What it is	Assistant platform and orchestration layer	Instruction-tuned model family
Unit of behavior	Session and workflow level	Single prompt/response completion
Tool execution	Owns tool policy and execution boundaries	Produces candidate tool-call content only
State continuity	Coordinates multi-turn memory and agent state	No durable state by itself
Operations scope	Gateway health, routing, policy, channels, fallback	Model quality, context behavior, latency, VRAM fit
Security boundary	Enforces allowlists/sandbox policy	Cannot enforce policy on its own

The table above is the key architectural point: OpenClaw is where you enforce behavior. Hermes is where you get language intelligence.

8.4 Tool-calling behavior and reliability

Hermes models are commonly chosen because they tend to be capable instruction followers and often behave well with structured output patterns. That can improve tool-call formatting quality in real workloads.

OpenClaw still must validate all tool-call payloads and enforce policy, because model output

is never a security control. Even with a strong model, malformed or unsafe calls can still appear. The robust pattern is model capability plus strict boundary checks, not model capability instead of boundary checks.

8.5 Context windows and long-horizon work

People often ask whether a stronger model family alone is enough for long-running assistant tasks. Usually it is not.

Long-horizon assistant behavior depends on two things:

1. Model context capacity and retrieval quality under load.
2. Orchestration discipline in what gets carried forward, summarized, or dropped.

Hermes helps with the first dimension. OpenClaw governs the second. For multi-file coding and operations workflows, the second dimension usually determines whether the system remains stable over time.

8.6 Deployment and day-2 operations

Running Hermes well is mostly a model-serving problem: quantization choice, GPU/CPU split, context sizing, and endpoint stability.

Running OpenClaw well is mostly an orchestration problem: channel policy, sandbox defaults, fallback chains, state backup, and recovery runbooks.

That split is operationally useful when debugging incidents:

1. If formatting degrades or reasoning quality shifts, inspect model/provider behavior first.
2. If tool scope, routing, or state continuity breaks, inspect OpenClaw policy and runtime state first.

8.7 Security and control-plane differences

OpenClaw can limit blast radius through non-main sandboxing, reduced workspace access, and constrained tool policy. Hermes cannot do that by itself because it has no direct permission system over host operations.

Treat every model output as untrusted input to the execution layer. The right place to enforce safety is the OpenClaw boundary that receives and validates candidate actions.

8.8 Benchmark-style comparison matrix

If you want to compare these two in a way that informs purchasing and deployment decisions, benchmark the stack as "OpenClaw plus model backend" versus "model-only direct usage" on the same hardware and prompt set.

The table below is not a universal scorecard. It is a practical measurement frame so teams can produce repeatable numbers in their own environment.

Measurement axis	Hermes direct (model-only)	OpenClaw + Hermes (or other backend)	Why it matters
First-token latency	Usually lower because there is minimal orchestration overhead	Usually higher due to routing, policy checks, and tool loop setup	Determines interaction feel for short requests
End-to-end task completion time	Fast for single-pass prompts	Usually better for multi-step tasks with tools because orchestration reduces retries	Captures real productivity, not just raw generation speed
Tokens per second (steady generation)	Primarily model/quantization bound	Similar model-bound throughput, but can include pauses around tool phases	Helps separate inference bottlenecks from agent-loop bottlenecks
VRAM and RAM footprint	Lower platform overhead; mostly serving stack	Higher total footprint due to gateway/session/tool processes	Drives hardware sizing and concurrency ceilings
Tool-call format pass rate	Not applicable without an orchestration layer	Critical metric: percent of tool calls that validate on first parse	Directly impacts reliability of agent workflows
Tool-call success rate	Not applicable without execution layer	Measures successful execution after validation and policy checks	Exposes integration breakage versus model-format issues

Measurement axis	Hermes direct (model-only)	OpenClaw + Hermes (or other backend)	Why it matters
Context retention quality over long tasks	Depends on model context behavior only	Depends on both model context and OpenClaw summarization/state policy	Predicts stability on long coding or operations sessions
Failure containment	Limited to prompt-level controls	Stronger when sandboxing, allowlists, and boundary checks are enforced	Defines blast radius under adversarial or malformed inputs
Recovery time after provider failure	Manual reroute or retry	Can be reduced with configured fallback chains and health-aware routing	Key for uptime and unattended workflows
Operational observability	Model logs and serving metrics	Model metrics plus gateway/session/tool lifecycle telemetry	Faster root-cause isolation in production-like setups

For reproducible results, run at least three workload classes with fixed prompts and a fixed hardware profile:

1. Single-shot generation (summaries, extraction, one-file edits).
2. Multi-step tool workflow (shell plus file edits plus validation).
3. Long-horizon session (multi-file changes over an extended context window).

Track medians and p95 values separately. A configuration can look good on average while still being painful in tail latency.

For tool-centric workflows, include two explicit quality metrics:

1. Tool-call schema pass rate on first attempt.
2. Task success without manual intervention.

Those two often predict user trust better than raw token throughput.

One practical reading rule helps avoid false conclusions: if model-only appears “faster” but

fails more multi-step tasks, it is usually optimized for benchmark shape rather than real workflow completion. For assistant operations, completion reliability is typically more valuable than isolated generation speed.

8.9 When to use each approach

Use Hermes directly when your task is mostly single-pass generation and you do not need channel routing, persistent assistant state, or orchestrated tools.

Use OpenClaw with Hermes when you want local reasoning quality plus durable assistant operations: channel integration, policy control, fallback behavior, and repeatable runbooks.

For most users in this primer's target audience, the practical goal is not choosing one over the other. It is composing them correctly:

1. OpenClaw as control plane.
2. Hermes as one backend option in model policy.
3. Hosted fallback only where reliability requirements justify it.

That architecture gives you local-first behavior without giving up operational resilience.

9 Podman + Local LLMs: Containment Patterns

There are three practical containment patterns.

Pattern one runs OpenClaw in containers but leaves model services on host. It is easy to adopt but less self-contained. Pattern two containerizes both gateway and model services with explicit persistence paths, which is often the best balance of containment and operability. Pattern three adds stricter sandboxing and narrow tool policies for higher-risk surfaces.

Most mature setups converge toward pattern two after proving behavior in pattern one.

10 Example Configuration Snippets

10.1 Local-first with hosted fallback

```
{
  agents: {
    defaults: {
      model: {
        primary: "ollama/gemma4",
        fallbacks: ["anthropic/claude-sonnet-4-6"]
      }
    }
  },
  models: {
```

```
    mode: "merge",
    providers: {
      ollama: {
        baseUrl: "http://ollama:11434",
        api: "ollama",
        apiKey: "ollama-local",
        timeoutSeconds: 300,
        models: [
          {
            id: "gemma4",
            name: "gemma4",
            reasoning: false, // no reasoning-token support in this API mode; model quality is unaffected
            input: ["text"],
            cost: { input: 0, output: 0, cacheRead: 0, cacheWrite: 0 },
            contextWindow: 65536,
            maxTokens: 4096
          }
        ]
      }
    }
  }
}
```

10.2 Non-main sandbox baseline

```
{
  agents: {
    defaults: {
      sandbox: {
        mode: "non-main",
        scope: "agent",
        workspaceAccess: "none"
      }
    }
  }
}
```

10.3 Generic OpenAI-compatible local provider

```
{
  agents: {
    defaults: {
      model: { primary: "local/my-model" }
    }
  },
  models: {
    mode: "merge",
    providers: {
      local: {
        baseUrl: "http://127.0.0.1:8000/v1",
        apiKey: "sk-local",
        api: "openai-completions",
        timeoutSeconds: 300,
        models: [
          {
            id: "my-model",
            name: "my-model",
            reasoning: false,
            input: ["text"],
            cost: { input: 0, output: 0, cacheRead: 0, cacheWrite: 0 },
```

```
            contextWindow: 120000,
            maxTokens: 8192
          }
        ]
      }
    }
  }
}
```

11 Operational Reference

11.1 Security and Hardening Checklist

Hardening should scale with exposure. Loopback-only personal setups can prioritize convenience. Any remotely reachable surface should prioritize strict channel policy, controlled tool access, and sandbox boundaries.

At minimum, keep publish scope narrow, enforce pairing and allowlists, avoid broad host binds, and run diagnostics after significant config changes.

11.2 Troubleshooting Guide

Start with transport and state truth before tuning behavior. Reachability failures usually come from runtime/port/publish issues. Auth failures usually come from token mismatch or target confusion. Provider mismatches often come from namespace assumptions in containerized environments.

When tool calls appear as plain text, treat backend compatibility as a likely cause before rewriting assistant logic.

11.3 Hardening Profile Matrix

Control Area	Dev	Trusted-Home	Internet-Exposed
Publish scope	loopback	loopback + controlled remote access	loopback + authenticated proxy/tailnet
Channel policy	minimal	pairing + allowlists	strict pairing + strict allowlists
Sandbox mode	off/non-main	non-main	all or tightly scoped non-main
Workspace access	rw acceptable	prefer none/ro	none by default

Control Area	Dev	Trusted-Home	Internet-Exposed
Tool policy	broad for testing	constrained	deny-by-default for risky tools
Fallback strategy	simple	explicit chain	explicit chain + active monitoring
Backup policy	ad hoc	scheduled	scheduled + off-host encrypted retention

Do not advance to a higher exposure profile until the current profile is stable and validated.

12 Reference Links

- OpenClaw repository: https://github.com/openclaw/openclaw
- OpenClaw docs: https://docs.openclaw.ai
- Podman install guide: https://docs.openclaw.ai/install/podman
- Docker guide: https://docs.openclaw.ai/install/docker
- Models: https://docs.openclaw.ai/concepts/models
- Model failover: https://docs.openclaw.ai/concepts/model-failover
- Local models: https://docs.openclaw.ai/gateway/local-models
- Local model services: https://docs.openclaw.ai/gateway/local-model-services
- Sandboxing: https://docs.openclaw.ai/gateway/sandboxing
- Ollama provider: https://docs.openclaw.ai/providers/ollama
- LM Studio provider: https://docs.openclaw.ai/providers/lmstudio
- vLLM provider: https://docs.openclaw.ai/providers/vllm
- LiteLLM provider: https://docs.openclaw.ai/providers/litellm

---

13 Runbooks

13.1 Full Podman Compose Stack (OpenClaw + Ollama + Optional vLLM)

This project includes a concrete compose baseline so the primer is directly actionable. The stack is designed for local-only exposure, explicit persistence, and optional model-serving expansion.

`podman-compose.yml` publishes two loopback-only ports on the `openclaw` service: `18789` for the dashboard covered earlier, and `18790` for the gateway's bridge connection. Both stay loopback-bound by default — treat any change that exposes `18790` beyond localhost with the same scrutiny as the dashboard port.

13.1.1 Included operational files

- `podman-compose.yml`
- `scripts/backup_state.sh`
- `scripts/restore_state.sh`

13.1.2 Launch flow

```
podman compose -f podman-compose.yml up -d
podman compose -f podman-compose.yml ps
podman compose -f podman-compose.yml logs -f openclaw
```

13.1.3 Optional vLLM profile

```
podman compose -f podman-compose.yml --profile vllm up -d
```

13.2 Backup and Restore

State integrity is central to reliable assistant operation. The included scripts provide a baseline snapshot and restore workflow.

13.2.1 Backup

```
./scripts/backup_state.sh
```

13.2.2 Restore

```
./scripts/restore_state.sh ./backups/openclaw_state_YYYYMMDD_HHMMSS.tar.gz
```

13.2.3 Post-restore validation

```
podman compose -f podman-compose.yml up -d
openclaw gateway status --deep
openclaw models status
openclaw models list --provider ollama
```

13.3 Deterministic Bring-Up Sequence

```
podman compose -f podman-compose.yml up -d
export OPENCLAW_CONTAINER=openclaw
openclaw onboard
openclaw models status
openclaw config set agents.defaults.sandbox.mode '"non-main"'
openclaw gateway status --deep
./scripts/backup_state.sh
```

14 Closing Perspective

The real value of this stack is not simply running local models. It is controlling assistant behavior under explicit operational rules you own. If you maintain clear boundaries for runtime, state, policy, and recovery, OpenClaw can move from "interesting tool" to dependable daily system.

No deployment is literally zero-touch. The real goal is explicit host contracts, explicit persistence, explicit secret handling, and explicit recovery steps. OpenClaw plus rootless Podman fits this model well when boundary discipline is maintained.